

Puesta en marcha del servidor

Planilla de Horas — despliegue con Docker en el servidor definitivo

Linux y Windows · HTTPS · Backups · Operación diaria

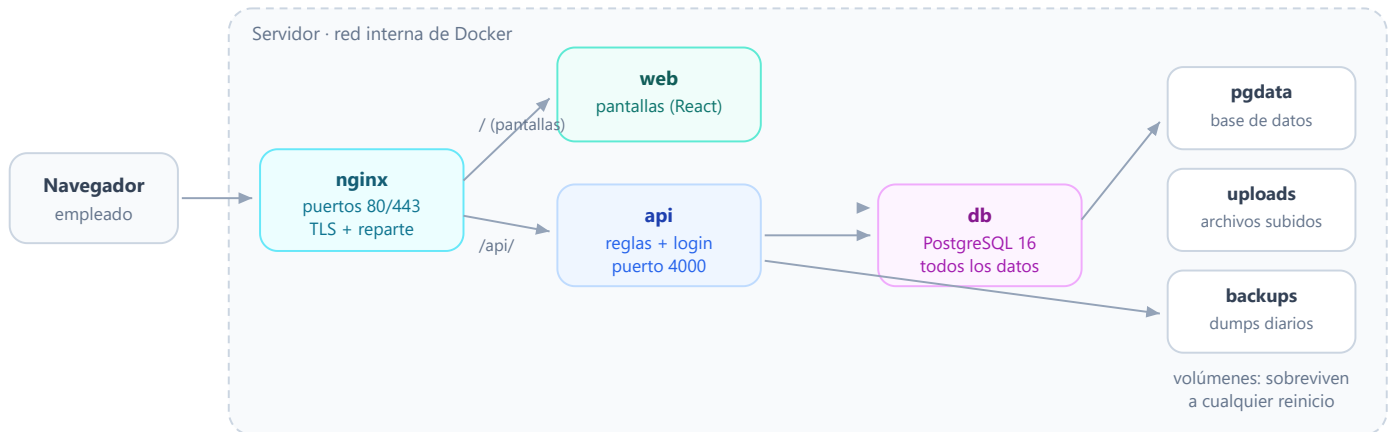
Contenido

- 1 · Qué se levanta
- 2 · Requisitos del servidor
- 3 · Cómo va a entrar la gente
- 4 · Instalación en Linux
- 5 · Instalación en Windows
- 6 · Publicar con HTTPS
- 7 · Primer ingreso
- 8 · Operación diaria
- 9 · Actualizar a una versión nueva
- 10 · Backups y restauración
- 11 · Problemas frecuentes
 - 11.1 · Aviso de servidor caído
 - 11.2 · Verificar CGNAT
- A · Variables del archivo .env
- B · Archivos de infraestructura

Esta guía asume que el servidor arranca **desde cero**. Todo el procedimiento está automatizado en un único archivo: `instalar-servidor.bat` en Windows y `deploy/bootstrap.sh` en Linux. Los capítulos 4 y 5 son ese archivo paso a paso, para saber qué está haciendo y poder intervenir si algo falla.

1 · Qué se levanta

La aplicación no es un solo programa: son **cuatro contenedores** que Docker arranca y conecta entre sí. Cada uno hace una cosa y solo el primero está expuesto a la red.



Los cuatro contenedores y los tres volúmenes de datos.

Contenedor	Qué hace	Expuesto
nginx	Única puerta de entrada. Termina el HTTPS y decide si el pedido va a las pantallas o al API.	Sí — puertos 80 y 443
web	Sirve las pantallas ya compiladas (HTML, JS, CSS). No tiene lógica ni datos.	No
api	Todas las reglas: login, planillas, aprobaciones, vacaciones, exportaciones. También genera el backup diario.	No
db	PostgreSQL 16. Guarda absolutamente todo.	No

Por qué importa que solo nginx esté expuesto: la base de datos no tiene puerto abierto hacia afuera. Aunque alguien conozca la IP del servidor, no puede conectarse a Postgres: solo existe dentro de la red privada de Docker.

Sin Redis. Las versiones anteriores de la configuración levantaban un contenedor Redis. El código de la aplicación nunca lo usó, así que se quitó: era memoria RAM ocupada sin función. Lo mismo con las claves VAPID de notificaciones push — las notificaciones funcionan por consulta periódica, no por push del navegador.

2 · Requisitos del servidor

Hardware mínimo

- **2 núcleos** de CPU
- **4 GB de RAM** (2 GB alcanza pero deja poco margen para la compilación de imágenes)
- **20 GB de disco** libres
- Conexión de red estable y, si va a entrar gente de afuera, IP alcanzable o túnel

Para ~100 empleados esto sobra. El consumo real en reposo ronda 1 GB de RAM.

Software

- **Docker Engine 24+** con el plugin `compose` v2 (Linux), o **Docker Desktop 4.30+** (Windows)
- **git** para traer el código (o una copia del proyecto en un pendrive)
- Nada más: Node.js, PostgreSQL y nginx **no** se instalan en el servidor, van adentro de los contenedores

2.1 · Qué sistema operativo conviene

Opción	Ventajas	Cosas a tener en cuenta
Ubuntu Server 24.04 LTS recomendado	Docker nativo y liviano; se administra por SSH; se reinicia solo tras un corte de luz; es el entorno donde más se prueba este tipo de stack.	Requiere manejarse por línea de comandos.
Windows 11 / Server + Docker Desktop	Entorno familiar, interfaz gráfica para ver contenedores y logs.	Docker Desktop corre sobre WSL2: consume ~2 GB de RAM extra. Hay que configurar el inicio automático y que la sesión de Windows quede iniciada.

Los dos caminos usan **exactamente los mismos archivos** del proyecto. La única diferencia es el script de arranque (`.sh` o `.ps1`) y cómo se instala Docker. Si mañana se cambia de Windows a Linux, la migración es copiar los backups y volver a correr el script.

3 · Cómo va a entrar la gente

Esta es la decisión que hay que tomar **antes** de instalar, porque cambia dos líneas del archivo de configuración. Todo lo demás es igual.

Opción	Costo	Abrir puertos	Cómo entra el usuario
DuckDNS + Let's Encrypt recomendado	Gratis	Sí (80 y 443)	https://wenlenplanilla.duckdns.org — dirección fija y gratuita, con certificado real. Se actualiza sola cuando el proveedor de internet cambia la IP.
Cloudflare Tunnel + dominio propio	~USD 12/año (el dominio)	No	https://planilla.tuempresa.com — la dirección más corta y presentable. El túnel sale desde el servidor: no hace falta tocar el router ni tener IP fija.
Dominio propio + Let's Encrypt	~USD 12/año	Sí (80 y 443)	https://planilla.tuempresa.com — el certificado lo emite y renueva certbot en el propio servidor. Requiere IP pública fija o DNS dinámico.
Solo red interna	Gratis	No	http://192.168.1.50 — solo desde la red de la oficina o por VPN. Sin certificados.

3.1 · Sobre el HTTPS: cuándo es obligatorio

Si la aplicación se va a usar **solo dentro de la oficina**, HTTP alcanza. En cuanto se accede desde afuera —celular, casa, obra— el HTTPS deja de ser opcional:

- El usuario y la contraseña viajan en cada ingreso. Sin cifrar, cualquiera en el camino (un WiFi público, un proveedor intermedio) los puede leer en texto plano.
- Chrome y Edge muestran el cartel **"No es seguro"** junto al formulario de acceso.
- Sin HTTPS el navegador bloquea la instalación de la app en el celular y las notificaciones del sistema.

Lo que no funciona: publicar la IP con un puerto (<http://186.x.x.x:3000>) y "disfrazarla" con un acortador de links. El acortador solo redirige: la barra del navegador termina mostrando la IP igual, y **Let's Encrypt no emite certificados para direcciones IP**, así que no hay forma de tener HTTPS por ese camino. Sumado a que la IP hogareña o de oficina suele cambiar sola cada vez que se reinicia el módem, dejando la app inaccesible sin aviso.

3.2 · Conseguir la dirección sin comprar nada

DuckDNS regala subdominios permanentes. El alta lleva dos minutos: entrar a duckdns.org, ingresar con una cuenta de Google o GitHub, elegir un nombre libre —por ejemplo [wenlenplanilla](https://wenlenplanilla.duckdns.org)— y copiar el token que muestra la página. Eso deja <https://wenlenplanilla.duckdns.org> reservado para siempre, sin costo ni tarjeta.

Sobre esa dirección se emite un certificado de Let's Encrypt igual que con un dominio pago: el candado del navegador es idéntico, no hay pantallas intermedias ni publicidad. La única diferencia real es estética — es más larga que un [.com](https://planilla.com) propio.

Lo que sí necesita: abrir los puertos 80 y 443 del router hacia el servidor, y que el proveedor de internet dé una IP pública real. Si la conexión está detrás de **CGNAT** —una IP compartida entre varios clientes— no hay puerto que abrir y esta opción no sirve; ahí la única salida es un túnel, que sí requiere dominio propio. Cómo verificarlo, en el capítulo 11.

Camino más corto para arrancar hoy: instalar con la opción **HTTP puro** (capítulo 4 o 5) y usar la aplicación en la red interna. Cuando la dirección de DuckDNS esté lista, se cambian dos líneas del `.env`, se emite el certificado y queda publicada. No hay que reinstalar nada ni se pierden datos.

4 · Instalación en Linux (Ubuntu / Debian)

Del sistema recién instalado a la aplicación funcionando. Todo por SSH.

1 Instalar Docker

```
# Script oficial de Docker: instala el motor y el plugin compose
curl -fsSL https://get.docker.com | sudo sh

# Permitir usar docker sin sudo (cerrar sesión y volver a entrar después)
sudo usermod -aG docker $USER

# Que arranque solo cuando se prende el servidor
sudo systemctl enable --now docker
```

Verificación: `docker compose version` debe responder `v2.x`.

2 Traer el proyecto

```
sudo mkdir -p /opt/planilla && sudo chown $USER /opt/planilla
git clone <URL-DEL-REPOSITORIO> /opt/planilla
cd /opt/planilla
```

Si no hay repositorio remoto, copiar la carpeta del proyecto por `scp` o pendrive. No hace falta traer `node_modules` ni `dist`: se generan al construir las imágenes.

3 Correr el script de puesta en marcha

```
bash deploy/bootstrap.sh
```

Va a preguntar dos cosas y hacer el resto solo:

- **URL pública** — `https://planilla.tuempresa.com`, o `http://192.168.1.50` si por ahora es solo red interna.
- **Modo de publicación** — `1` para HTTPS con dominio propio, `2` para HTTP puro (LAN o detrás de un túnel).

Después, sin intervención:

- Genera el archivo `.env` con contraseñas aleatorias (permisos `600`).
- Construye las imágenes de `api` y `web` — la primera vez son entre 5 y 15 minutos según la conexión.
- Levanta los cuatro contenedores.
- Aplica las migraciones de la base de datos.
- Espera a que el API conteste sano.

4 Cargar los datos iniciales

Si la base está vacía, el script pregunta si cargar el **seed**: crea la empresa, los sectores, los flujos de aprobación, los feriados y la nómina de empleados. Responder **s** la primera vez. En cualquier reinstalación posterior sobre datos existentes, responder **N**.

El seed **solo debe correrse una vez**, sobre una base vacía. Si se corre de nuevo con datos cargados, falla por registros duplicados (no borra nada, pero deja el proceso a medias).

5 Emitir el certificado (solo si se eligió HTTPS)

Hay un orden obligatorio acá: nginx **no arranca** si el certificado no existe, pero Let's Encrypt necesita que nginx esté sirviendo el puerto 80 para verificar el dominio. Por eso el script deja el stack en modo HTTP y después se pasa a HTTPS:

```
# 1. Emitir el certificado (el stack ya está arriba en modo HTTP)
docker compose -f docker-compose.prod.yml --profile certbot run --rm certbot \
  certonly --webroot -w /var/www/certbot \
  -d planilla.tuempresa.com --agree-tos -m admin@tuempresa.com

# 2. Cambiar el modo en el archivo .env
#   NGINX_CONF=./nginx.http.conf → NGINX_CONF=./nginx.conf

# 3. Recrear nginx, ahora con TLS
docker compose -f docker-compose.prod.yml up -d nginx
```

Requisito: el dominio ya debe apuntar a la IP del servidor y el puerto 80 estar accesible desde internet — así verifica Let's Encrypt que el dominio es tuyo. Si el paso 1 falla, casi siempre es porque el DNS todavía no propagó o el router no reenvía el puerto 80.

4.1 · Que arranque solo después de un corte de luz

Ya está resuelto: los contenedores tienen `restart: always` y Docker está habilitado en systemd. Cuando el servidor vuelve, la aplicación vuelve sola. Para comprobarlo: `sudo reboot` y verificar a los dos minutos que la página responde.

5 · Instalación en Windows

Mismo resultado que en Linux, con Docker Desktop sobre WSL2. Acá hay un **único archivo para ejecutar** — `instalar-servidor.bat` — que verifica la máquina, instala lo que falte y deja la aplicación funcionando.

1 Copiar el proyecto a la máquina

```
cd C:\
git clone <URL-DEL-REPOSITORIO> C:\planilla
```

O copiar la carpeta con un pendrive. Evitar rutas con espacios o acentos, y no ponerlo en OneDrive: la sincronización interfiere con los archivos que Docker monta.

2 Ejecutar `instalar-servidor.bat`

Doble clic sobre el archivo, en la raíz del proyecto. Es lo único que hay que ejecutar: el asistente se encarga del resto.

Donde se va a instalar el servidor?

- [1] En ESTA máquina Windows
- [2] En un servidor Linux (te doy el comando exacto)
- [3] Solo verificar requisitos (no instala nada)
- [4] Salir

Conviene empezar por la **opción 3**: revisa la máquina sin tocar nada y muestra qué falta. Verifica versión de Windows, arquitectura, memoria, espacio en disco, virtualización habilitada en la BIOS, salida a internet, puertos 80 y 443 libres, y si Docker está instalado y corriendo.

3 Elegir la opción 1 e ir contestando

Con los requisitos en orden, el asistente:

- Ofrece **instalar Docker Desktop** si falta (lo baja con winget). Después de eso hay que abrirlo una vez, esperar a que diga **“Engine running”** y volver a ejecutar el instalador.
- Pregunta la **dirección pública**: DuckDNS —pide subdominio y token— o red interna, en cuyo caso propone la IP local detectada.
- Genera el `.env` con claves aleatorias.
- Construye las imágenes y levanta los cuatro contenedores.
- Espera a que la aplicación responda y ofrece cargar los **datos iniciales**.
- Deja Docker Desktop configurado para arrancar con el equipo.
- Al final muestra la dirección, las credenciales de ingreso y —si eligió DuckDNS— el comando exacto para emitir el certificado.

Es seguro volver a ejecutarlo: si ya existe un `.env`, lo conserva y solo actualiza la aplicación. No se pierden datos.

4 Abrir los puertos en el Firewall de Windows

```
# En PowerShell como administrador
New-NetFirewallRule -DisplayName "Planilla HTTP" -Direction Inbound -Protocol TCP -LocalPort 80 -Action Allow
New-NetFirewallRule -DisplayName "Planilla HTTPS" -Direction Inbound -Protocol TCP -LocalPort 443 -Action Allow
```

Además hay que reenviar esos mismos puertos en el router hacia esta máquina. Innecesario si se usa Cloudflare Tunnel: ahí la conexión sale desde el servidor y no entra nada.

El punto flojo de Windows: Docker Desktop necesita que haya una sesión de usuario iniciada. Si el servidor se reinicia y queda en la pantalla de bloqueo sin que nadie entre, la aplicación **no levanta**. Configurar el inicio de sesión automático (netplwiz → destildar “Los usuarios deben escribir su nombre y contraseña”) y acostumbrarse a **bloquear** la pantalla en lugar de cerrar sesión.

Certificados en Windows: el comando de certbot del capítulo 6.1 funciona igual, porque Docker es el mismo. Cambia solo el carácter de continuación de línea: en PowerShell es el acento grave ` en lugar de la barra \.

6 · Publicar con HTTPS

6.1 · DuckDNS + Let's Encrypt dirección fija, gratis

El camino sin costo: una dirección permanente propia y un certificado real. Requiere abrir dos puertos en el router.

1 Reservar la dirección

En duckdns.org, ingresar con Google o GitHub, escribir el nombre deseado y presionar **add domain**. Anotar el **token** que aparece arriba de la página: es la credencial de la cuenta.

2 Reenviar los puertos en el router

En la configuración del router, sección *Port forwarding* (o *Redirección de puertos / Servidores virtuales*), crear dos reglas hacia la IP local del servidor: **80 → 80** y **443 → 443**, protocolo TCP. Conviene además reservarle esa IP al servidor en el DHCP para que no cambie sola.

3 Configurar el stack

En el archivo `.env`:

```
DUCKDNS_SUBDOMAIN=wenlenplanilla
DUCKDNS_TOKEN=<el token de duckdns.org>
PUBLIC_URL=https://wenlenplanilla.duckdns.org
NGINX_CONF=./nginx.http.conf # temporal: hasta emitir el certificado
```

```
docker compose -f docker-compose.prod.yml --profile duckdns up -d
```

El contenedor `duckdns` queda corriendo y avisa la IP pública cada cinco minutos, así la dirección sigue funcionando aunque el proveedor la cambie.

4 Emitir el certificado y pasar a HTTPS

```
docker compose -f docker-compose.prod.yml --profile certbot run --rm certbot \
  certonly --webroot -w /var/www/certbot \
  -d wenlenplanilla.duckdns.org --agree-tos -m TU_EMAIL

# Cambiar en .env: NGINX_CONF=./nginx.conf
docker compose -f docker-compose.prod.yml up -d nginx
```

La renovación automática es la misma del punto 6.3 — no es opcional.

Para verificar que quedó bien, probar la dirección desde un celular **con los datos móviles**, no desde el WiFi de la oficina: así se comprueba que entra de verdad desde afuera y no por la red interna.

Dos direcciones: pruebas y producción

DuckDNS permite varios subdominios en la misma cuenta, así que conviene separarlos:

Dirección	Apunta a	Uso
<code>planillawenlen.duckdns.org</code>	Servidor Docker	La que usa la empresa. Puertos 80 y 443, con HTTPS.
<code>planillawenltest.duckdns.org</code>	PC de pruebas (<code>start-remote.bat</code>)	Para probar cambios antes de publicarlos. Se configura en <code>deploy/tunnel.env</code> .

Ojo con el choque de puertos. Los dos nombres resuelven a la **misma IP pública** —la de la oficina— y el router solo puede reenviar cada puerto a una máquina. Si el servidor ya se quedó con el 80 y el 443, la PC de pruebas necesita otro puerto: se reenvía, por ejemplo, el **8080 al 3000** de esa PC, y la dirección de pruebas queda `http://planillawenltest.duckdns.org:8080`.

El entorno de pruebas queda en **HTTP sin certificado**: lo levanta Vite directamente, sin nginx que termine el TLS. Para probar internamente alcanza, pero el navegador va a marcar “No es seguro” y la aplicación no se puede instalar como PWA. Si hace falta probar exactamente lo que van a ver los usuarios, conviene hacerlo contra el servidor de producción.

6.2 · Cloudflare Tunnel requiere dominio propio

Es el camino más simple cuando no hay IP fija ni acceso al router: el servidor abre una conexión **de salida** hacia Cloudflare y por ahí entra el tráfico. La dirección — `https://planilla.tuempresa.com` — queda fija para siempre y no depende de que la IP de la conexión cambie.

Diferencia con el túnel de las pruebas. El que se viene usando en el testing es un *quick tunnel*: gratis y sin cuenta, pero genera una dirección al azar (algo-raro-1234.trycloudflare.com) que **cambia cada vez que el proceso arranca**. El túnel *con nombre* que se describe acá es el mismo programa, pero registrado en una cuenta: siempre la misma URL, sobreviva lo que sobreviva.

Paso a paso, una sola vez

1 Comprar el dominio

En Cloudflare Registrar o Namecheap (`.com` , ~USD 10–12 por año) o en NIC Argentina (`.com.ar`). Con el dominio comprado ya se define cómo va a entrar la gente: `planilla.tuempresa.com` es más corto y claro que cualquier alternativa gratuita.

2 Agregar el dominio a Cloudflare

Cuenta gratuita en `cloudflare.com` → **Add a site**. Cloudflare devuelve dos servidores DNS que hay que cargar en el panel del registrador donde se compró el dominio. La activación tarda entre unos minutos y unas horas.

3 Crear el túnel y copiar el token

En `one.dash.cloudflare.com` → **Networks** → **Tunnels** → **Create a tunnel**, tipo **Cloudflared**, con un nombre cualquiera (por ejemplo `planilla-wenlen`). La pantalla siguiente muestra un comando con un **token** largo que empieza con `eyJ...` — ese token es la credencial del túnel.

4 Definir a dónde apunta

En la pestaña **Public Hostnames** del túnel, **Add a public hostname**:

Campo	Servidor Docker	PC de pruebas
Subdomain	<code>planilla</code>	
Domain	el dominio comprado	
Type	<code>HTTP</code>	
URL	<code>nginx:80</code>	<code>localhost:3000</code>

5 Conectar el token

En el servidor **Docker**, en el archivo `.env` :

```
CLOUDFLARE_TUNNEL_TOKEN=eyJ...
NGINX_CONF=./nginx.http.conf # el HTTPS lo pone Cloudflare
PUBLIC_URL=https://planilla.tuempresa.com
```

```
docker compose -f docker-compose.prod.yml --profile tunnel up -d
```

En la **PC de pruebas** (mientras el servidor definitivo no exista), copiar `deploy\tunnel.env.example` a `deploy\tunnel.env` y completar las dos líneas. `start-remote.bat` lo detecta solo: usa el túnel con nombre, autoriza el dominio en el API y muestra la URL fija en lugar de la aleatoria. Si ese archivo no existe, se comporta exactamente como antes.

El mismo dominio sirve para las dos etapas. Durante las pruebas apunta a la PC de testing; cuando el servidor definitivo esté listo, se cambia el destino del hostname en el panel de Cloudflare (de `localhost:3000` a `nginx:80`) y la gente sigue entrando por el mismo link, sin enterarse de la mudanza.

El token es una credencial. Quien lo tenga puede publicar un túnel en nombre de ese dominio. No se sube a git (`deploy/tunnel.env` está excluido) ni se comparte por chat. Si se filtra, se revoca desde el panel de Cloudflare y se genera uno nuevo.

6.3 · Renovación de los certificados

La emisión inicial está en el capítulo 6.1 (o en el 4, paso 5): emitir en modo HTTP y recién después pasar a `nginx.conf` . La **renovación** hay que dejarla programada, porque los certificados de Let's Encrypt duran 90 días y esto vale igual para DuckDNS que para un dominio propio:

```
# Linux - agregar a la tabla de tareas: crontab -e
0 3 * * 1 cd /opt/planilla && docker compose -f docker-compose.prod.yml --profile certbot run --rm certbot renew --quiet &&
docker compose -f docker-compose.prod.yml restart nginx
```

```
# Windows - crear la tarea programada (PowerShell como administrador)
$a = New-ScheduledTaskAction -Execute 'powershell.exe' -Argument '-Command "cd C:\planilla; docker compose -f docker-
compose.prod.yml --profile certbot run --rm certbot renew --quiet; docker compose -f docker-compose.prod.yml restart nginx"'
$t = New-ScheduledTaskTrigger -Weekly -DaysOfWeek Monday -At 3am
Register-ScheduledTask -TaskName "Planilla - renovar certificado" -Action $a -Trigger $t -RunLevel Highest
```

Si el certificado vence, **nginx deja de arrancar** y la aplicación queda inaccesible aunque todo lo demás funcione. La tarea de renovación no es opcional: verificar una vez que corrió bien.

6.4 · Solo red interna

Con `NGINX_CONF=./nginx.http.conf` y `PUBLIC_URL=http://192.168.1.50` la aplicación queda disponible en la LAN sin certificados. Conviene reservar esa IP en el router (DHCP estático) para que no cambie sola y deje de funcionar el acceso.

7 · Primer ingreso

Con el stack levantado y el seed cargado, ya se puede entrar.

Cuenta de administración

Usuario `admin@wenlen.com`

Contraseña `Admin2026!`

Es la cuenta de sistema (rol ADMIN, sin sector). La aplicación exige cambiar la contraseña en el primer ingreso.

Empleados de la nómina

El seed carga a cada empleado con su correo corporativo y la contraseña inicial `Wenlen2026!`, también con cambio obligatorio en el primer ingreso.

Antes de dar la URL a la empresa: cambiar la contraseña del administrador y verificar que el archivo `.env` no quedó copiado en ningún lado accesible. Ese archivo contiene la contraseña de la base de datos y las claves con las que se firman las sesiones: quien lo tenga puede entrar como cualquier usuario.

7.1 · Verificación rápida

Qué probar	Resultado esperado
Abrir la URL pública	Aparece la pantalla de ingreso, con candado si es HTTPS
Entrar como administrador	Pide cambiar la contraseña y entra al panel
<code>/api/v1/health</code> en el navegador	<code>{"status":"ok", ...}</code>
Crear una planilla y enviarla	Cambia de estado y le aparece al aprobador
Reiniciar el servidor	A los dos minutos la aplicación responde sola

8 · Operación diaria

Todos los comandos se ejecutan parado en la carpeta del proyecto (`/opt/planilla` o `C:\planilla`).

Para	Comando
Ver si está todo arriba	<code>docker compose -f docker-compose.prod.yml ps</code>
Ver los registros del API en vivo	<code>docker compose -f docker-compose.prod.yml logs -f api</code>
Ver los últimos 200 renglones	<code>docker compose -f docker-compose.prod.yml logs --tail 200 api</code>
Reiniciar un contenedor	<code>docker compose -f docker-compose.prod.yml restart api</code>
Apagar todo (los datos quedan)	<code>docker compose -f docker-compose.prod.yml down</code>
Volver a levantar	<code>docker compose -f docker-compose.prod.yml up -d</code>
Consumo de recursos	<code>docker stats</code>
Entrar a la base con SQL	<code>docker compose -f docker-compose.prod.yml exec db psql -U planilla_user -d planilla_horas</code>

Atajo: en Linux, agregar alias `dcp='docker compose -f /opt/planilla/docker-compose.prod.yml'` al archivo `~/.bashrc`. Después alcanza con `dcp ps`, `dcp logs -f api`.

El único comando peligroso: `docker compose down -v`. La `-v` borra los volúmenes, es decir **toda la base de datos y los archivos subidos**, sin confirmación ni papelera. Nunca usarlo en el servidor de producción.

9 · Actualizar a una versión nueva

Cuando hay cambios en el código, el procedimiento es siempre el mismo y no toca los datos.

```
cd /opt/planilla                # o cd C:\planilla

# 1. Respaldo previo - siempre, aunque el cambio parezca chico
docker compose -f docker-compose.prod.yml exec -T db \
  pg_dump -U planilla_user -Fc planilla_horas > respaldo-antes-de-actualizar.dump

# 2. Traer el código nuevo
git pull

# 3. Reconstruir y relevar (solo se recrea lo que cambió)
docker compose -f docker-compose.prod.yml up -d --build

# 4. Verificar
docker compose -f docker-compose.prod.yml ps
docker compose -f docker-compose.prod.yml logs --tail 50 api
```

Las **migraciones de base de datos** se aplican solas: el contenedor del API las corre al arrancar, antes de aceptar pedidos. La interrupción del servicio es de menos de un minuto.

Si algo sale mal: volver a la versión anterior con `git checkout <commit>` y repetir el paso 3. Si la migración ya modificó la base y hay que retroceder de verdad, restaurar el respaldo del paso 1 con el procedimiento del capítulo 10.

10 · Backups y restauración

10.1 · Lo que ya hace solo

El API genera un respaldo completo de la base **todos los días** y guarda las últimas **14 copias**, en dos carpetas separadas (`primary` y `secondary`) dentro del volumen `planilla_backups` . Es automático, no hay que configurar nada.

Eso no alcanza como respaldo real. Esas copias viven en el mismo disco del mismo servidor: si el disco falla o alguien borra el volumen, se pierden junto con la base. Un respaldo solo cuenta cuando está **en otra máquina**.

10.2 · Sacar los respaldos del servidor

```
# Copiar todos los dumps del volumen a un archivo comprimido
docker run --rm -v planilla_backups:/b -v $PWD:/out alpine \
  tar czf /out/planilla-backups-$(date +%F).tgz /b

# Y llevarlo a otra máquina (ejemplo con scp)
scp planilla-backups-*.tgz usuario@otra-maquina:/respaldos/
```

En Windows, reemplazar `$PWD` por `${PWD}` y `$(date +%F)` por una fecha escrita a mano.

Conviene dejarlo programado una vez por semana, con el mismo mecanismo del capítulo 6.3 (`crontab` en Linux, Tareas Programadas en Windows), y llevar la copia a un disco externo o a la nube.

10.3 · Restaurar

```
# 1. Ver qué respaldos hay disponibles
docker compose -f docker-compose.prod.yml exec api ls -lh backups/primary

# 2. Bajar el API para que nadie escriba durante la restauración
docker compose -f docker-compose.prod.yml stop api

# 3. Restaurar (--clean borra los objetos previos antes de recrearlos)
docker compose -f docker-compose.prod.yml exec -T db \
  pg_restore -U planilla_user -d planilla_horas --clean --if-exists \
  < respaldo-antes-de-actualizar.dump

# 4. Volver a levantar
docker compose -f docker-compose.prod.yml start api
```

Probar la restauración antes de necesitarla. Un respaldo que nunca se restauró es una suposición, no un respaldo. Conviene hacer el ejercicio completo una vez, idealmente en otra máquina, y anotar cuánto tardó.

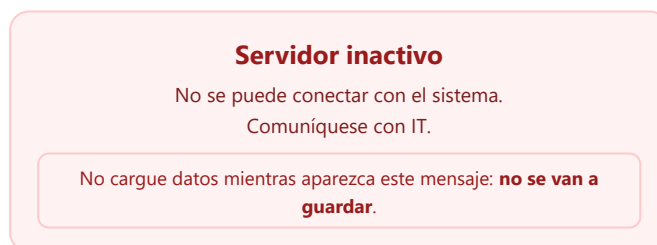
11 · Problemas frecuentes

Síntoma	Causa habitual	Qué hacer
La página no carga y <code>ps</code> muestra nginx en <code>Restarting</code>	Falta el certificado, o el dominio del <code>nginx.conf</code> no coincide con el real	Ver <code>logs nginx</code> . Emitir el certificado (cap. 6.1) o pasar temporalmente a <code>NGINX_CONF=/nginx.http.conf</code>
El API reinicia en bucle	No puede conectarse a la base, o falta una variable en el <code>.env</code>	<code>logs api</code> dice cuál. Verificar que <code>db</code> esté <code>healthy</code> y que el <code>.env</code> tenga los tres secretos
Entra pero al hacer login dice error de CORS	<code>PUBLIC_URL</code> no coincide exactamente con la URL usada	Debe ser idéntica, incluyendo <code>http/https</code> y sin barra final. Corregir el <code>.env</code> y <code>restart api</code>
<code>port is already allocated</code> al levantar	Otro programa ocupa el 80 o el 443 (IIS, Skype, otro nginx)	Liberarlo, o cambiar <code>HTTP_PORT / HTTPS_PORT</code> en el <code>.env</code>
La construcción de la imagen falla sin red	El servidor no llega a Docker Hub o al repositorio de PostgreSQL	Verificar DNS y proxy de salida. Las imágenes se bajan una sola vez
Los backups automáticos no aparecen	El volumen <code>backups</code> no está montado o quedó sin permisos	<code>exec api ls -l backups</code> . Deben existir <code>primary</code> y <code>secondary</code> con dueño <code>node</code>
Todo funciona pero está lento	Poca RAM: el sistema empieza a usar disco como memoria	<code>docker stats</code> . Si la suma se acerca al total del servidor, sumar RAM
(Windows) Después de reiniciar, nada levanta	Docker Desktop no arrancó porque no hay sesión iniciada	Configurar inicio de sesión automático y el arranque de Docker Desktop (cap. 5)

Lo primero, siempre: `docker compose -f docker-compose.prod.yml ps` para ver quién está caído, y `logs --tail 100 <servicio>` para leer el error concreto. El 90 % de los problemas dice exactamente qué pasa en esas dos salidas.

11.1 · Qué ve el empleado cuando el servidor está caído

La aplicación avisa sola. Si el servidor no responde —apagado, sin internet, corte de luz— a quien la tenga abierta le aparece un cartel que **bloquea la pantalla**:



Funciona **aunque el servidor esté completamente apagado**, porque quien lo muestra es la propia aplicación instalada en el dispositivo del empleado. Reintenta cada 10 segundos y, cuando el servidor vuelve, el cartel se cierra solo: lo que la persona tenía a medio escribir sigue ahí y puede guardarlo. Nadie pierde trabajo por cargar datos a ciegas.

Límite: alcanza a quien ya entró alguna vez desde ese dispositivo. Si alguien nunca abrió la aplicación y el servidor está caído, va a ver el error genérico del navegador —no hay nada instalado todavía que pueda mostrarle el mensaje.

11.2 · Verificar si la conexión está detrás de CGNAT

Determina si se pueden abrir puertos (capítulo 6.1) o si hace falta un túnel. Son dos datos que tienen que coincidir:

1. Buscar "*cuál es mi IP*" en Google desde una máquina de esa red y anotar el número.
2. Entrar al panel del router y buscar la **IP WAN** (o *IP de internet*).

Si **coinciden**, hay IP pública real y se pueden abrir puertos. Si la del router es distinta —sobre todo si empieza con `100.64.` hasta `100.127.`— la conexión está detrás de CGNAT: no hay puerto que abrir. En ese caso queda pedirle al proveedor una IP pública (a veces es gratis, a veces tiene un cargo) o usar un túnel.

Anexo A · Variables del archivo .env

Vive en la raíz del proyecto, junto a `docker-compose.prod.yml`. Lo genera el script de arranque a partir de `.env.docker.example`. Nunca se sube a git.

Variable	Obligatoria	Para qué sirve
<code>DB_PASSWORD</code>	Sí	Contraseña de PostgreSQL. La genera el script al azar.
<code>JWT_SECRET</code>	Sí	Clave con la que se firman las sesiones. Si cambia, se cierran todas las sesiones abiertas.
<code>JWT_REFRESH_SECRET</code>	Sí	Ídem para la renovación automática de sesión. Debe ser distinta de la anterior.
<code>PUBLIC_URL</code>	Sí	URL exacta con la que entran los usuarios. Define qué origen acepta el API (CORS) y los links de los correos.
<code>NGINX_CONF</code>	Sí	<code>./nginx.conf</code> para HTTPS propio, <code>./nginx.http.conf</code> para HTTP puro.
<code>HTTP_PORT</code> / <code>HTTPS_PORT</code>	No	Puertos del servidor. Por omisión 80 y 443.
<code>TZ</code>	No	Zona horaria de los contenedores. Por omisión <code>America/Argentina/Buenos_Aires</code> .
<code>DUCKDNS_SUBDOMAIN</code> / <code>DUCKDNS_TOKEN</code>	No	Solo si se publica con DuckDNS (perfil <code>duckdns</code>). Mantienen la dirección apuntando a la IP actual.
<code>CLOUDFLARE_TUNNEL_TOKEN</code>	No	Solo si se publica con Cloudflare Tunnel.
<code>SMTP_*</code>	No	Servidor de correo para las notificaciones por email. Sin esto, las notificaciones se ven solo dentro de la aplicación.

Guardar una copia del .env fuera del servidor, en un gestor de contraseñas. Si se pierde y hay que reinstalar, la base de datos existente queda inaccesible: la contraseña de Postgres está solo ahí.

Anexo B · Archivos de infraestructura

Qué es cada archivo del repositorio que interviene en el despliegue.

Archivo	Función
<code>docker-compose.prod.yml</code>	Define los cuatro contenedores de producción, los volúmenes y los perfiles opcionales (<code>certbot</code> , <code>duckdns</code> , <code>tunnel</code>).
<code>docker-compose.yml</code>	Versión de desarrollo, con recarga en caliente. No se usa en el servidor.
<code>nginx.conf</code>	Configuración del nginx de entrada en modo HTTPS.
<code>nginx.http.conf</code>	Ídem en modo HTTP puro (LAN o detrás de un túnel).
<code>.env.docker.example</code>	Plantilla de variables. Se copia a <code>.env</code> y se completa.
<code>instalar-servidor.bat</code>	El archivo a ejecutar en Windows. Verifica requisitos, instala Docker si falta, configura la dirección, levanta todo y carga los datos iniciales. También tiene un modo que solo verifica, sin instalar nada.
<code>deploy/bootstrap.sh</code>	El equivalente para Linux.
<code>deploy/tunnel.env.example</code>	Plantilla para la URL fija con Cloudflare Named Tunnel, con el instructivo completo adentro. Se copia a <code>deploy/tunnel.env</code> (excluido de git) y lo lee <code>start-remote.bat</code> .
<code>apps/api/Dockerfile</code>	Imagen del API: compila TypeScript, genera el cliente de base de datos e incluye las herramientas de respaldo de PostgreSQL 16.
<code>apps/api/docker-entrypoint.sh</code>	Al arrancar: espera a la base, aplica las migraciones pendientes y recién ahí levanta el servicio.
<code>apps/web/Dockerfile</code>	Imagen de las pantallas: compila con Vite y las sirve como archivos estáticos.
<code>apps/web/nginx.web.conf</code>	Configuración interna de ese contenedor (rutas de la aplicación, caché de recursos).
<code>apps/web/public/sw.js</code>	Service worker: cachea la aplicación para que abra sin conexión y responde el aviso interno cuando el servidor no contesta (capítulo 11.1).
<code>apps/web/src/components/ServidorInactivo.tsx</code>	El cartel bloqueante de "Servidor inactivo", con reintento automático cada 10 segundos.

Detalle a tener presente: las pantallas se compilan llamando al API con **rutas relativas**. Eso hace que la misma imagen sirva para cualquier dominio: si mañana cambia la URL, no hay que reconstruir nada, solo actualizar `PUBLIC_URL` en el `.env` y reiniciar.

Estado de verificación de esta guía. Los archivos de infraestructura se escribieron a partir del código real del proyecto y se validaron sintácticamente (compose, scripts de Linux y Windows), y las compilaciones del API y de las pantallas se ejecutaron con éxito fuera de Docker. El aviso de servidor caído del capítulo 11.1 **sí** se probó de punta a punta: aplicación cargada, servidor apagado de verdad, cartel apareciendo al intentar ingresar, y cierre automático al volver el servicio. La construcción de las imágenes Docker, en cambio, **no** pudo probarse porque el equipo donde se preparó esto no tiene Docker instalado: la primera corrida real de bootstrap conviene hacerla con tiempo y mirando los registros.